

SD2125 Signals and Mechanical Systems

HOME ASSIGNMENT 2 (HA2)

STUDENT 1 Marcus von Post Carlsson

STUDENT 2 Spencer Karlström Huerta

In these exercises you are intended to learn some basic concepts in signal analysis and how to use these functions in MATLAB with. The assignment is divided into four main sections: (1) Fourier series, (2) sampling, (3) aliasing, and (4) spectrograms. The assignment is intended to be in pairs, but it is also fine to do it alone (in that case, please remove the STUDENT 2 line).

The product help under the help menu in MATLAB is recommended to use when understanding what predefined functions are suitable to implement. Pay special attention to how MATLAB outputs are organized. It is recommended to "Run Section" instead of "Run" (the whole code) to prevent unnecessary code runs.

INSTRUCTIONS FOR ASSIGNMENT AND REPORT

In this Live Script, you will find **empty CODE slots** where you must fill in relevant codes and **empty ANSWER slots** where you must fill in answers to given questions. The structure of the Live Script should not be modified, that is, **do not change/remove headings, numberings, or sections—even if you submit blank in any of them**. Make sure you also add your name(s) at the top of this document (here). If you did the assignment alone, add your name in Student 1 and delete the row for Student 2.

Once you are finished with your assignment, you should produce a PDF report out of the Live Script. No introduction, discussion nor conclusion are required in the report, only answer the questions and show the plots as instructed. You can produce the PDF report by clicking `"Save As..."` and selecting **PDF (*.pdf)** format.

OBS: **Make sure you save the PDF file including the name(s) of the student(s) who did the work: HA2_Initial-Surname_Initial-Surname, for example: `"HA2_K-Bolin_E-Zea.pdf"`.**

Table of Contents

SD2125 Signals and Mechanical Systems 1

HOME ASSIGNMENT 2 (HA2) 1
STUDENT 1 Marcus von Post Carlsson 1
STUDENT 2 Spencer Karlström Huerta 1
INSTRUCTIONS FOR ASSIGNMENT AND REPORT 1
1: FOURIER ANALYSIS 2
1.1 Fourier series 2
1.1.1. Fourier series of a harmonic signal 2
1.1.2. Fourier series of a sound signal 5
1.1.3. Explain the differences in the results seen in Tasks 1.1.1. and 1.1.2. 7
1.2 Fourier transform and inverse Fourier transform 7
1.2.1. Plot the time signal x with the correct x- and y-axis 8
1.2.2. Plot the raw and scaled output vector of the FFT magnitude spectrum 9
1.2.3. Describe how the output vector y from fft is sorted 10
1.2.4. Verify the inverse FFT of a forward FFT 10
1.2.5. Read and FFT the file "Classic.wav" with the code you used in Task 1.2.2, set n = 1024 11
2: SAMPLING 13
2.1. Read the Sound4.wav 4 s sample using “audioread” 13
2.2. Resample the signal at every 4th and 32nd points 15
2.3. Plot one-sided spectra using 512 samples for all three sampling frequencies (Fs, Fs/4, Fs/32) and compute the frequency resolution (Δf) for all these spectra 16
2.4. Listen to the downsampled signals and compare them to the original recording 18
3: ALIASES 18
3.1. Sample the sinusoid with 32 Hz, 20 Hz, 16 Hz, and 8 Hz 19
4: SPECTROGRAM 27
4.1. Plot the spectrogram for varying window size 28
4.2. Zoom-in and examine the spectrogram 31
4.3. Use the Doppler shift in the spectrogram to estimate the aircraft’s speed 31

1: FOURIER ANALYSIS

1.1 Fourier series

Discuss with your colleague the basic concept of Fourier series and what kind of signals they are suitable to map.

Signals must be periodic. Sum of additions of sinusoids and cosinuoids with different frequencies to match another periodic signal.

1.1.1. Fourier series of a harmonic signal

Calculate the Fourier series for the signal `sin_4_harmonic` below using the command “`fit`”. The Fourier series coefficients should be varied between `N = 1, 2, 4 & 8` see example below. Examine the coefficients in cN and the root mean squared error (rmse) in gN. Plot the results and compare to the original signal.

```
time = 0.001*[1:1000]';
sin_4_harmonic = 1*sin(time*4*pi) + 0.25*sin(time*4*4*pi);
[cN,gN] = fit(time, sin_4_harmonic,'fourierN');
```

```
% add your code here
T = (0.001:0.001:1)';
```

```
Nns = [1,2,4,8];

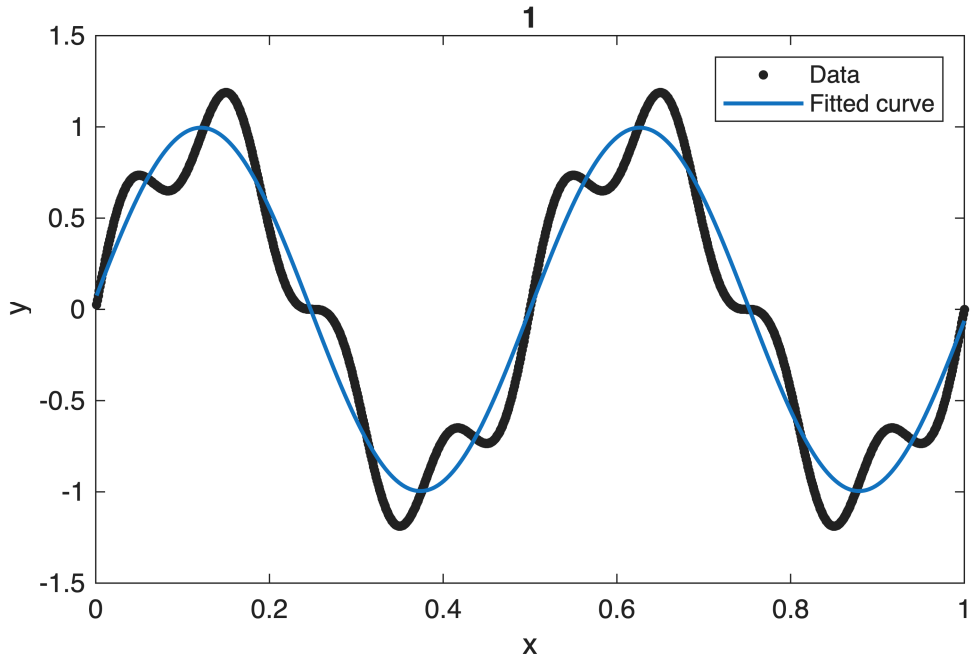
cN = cell(1, length(Nns));
gN = cell(1, length(Nns));

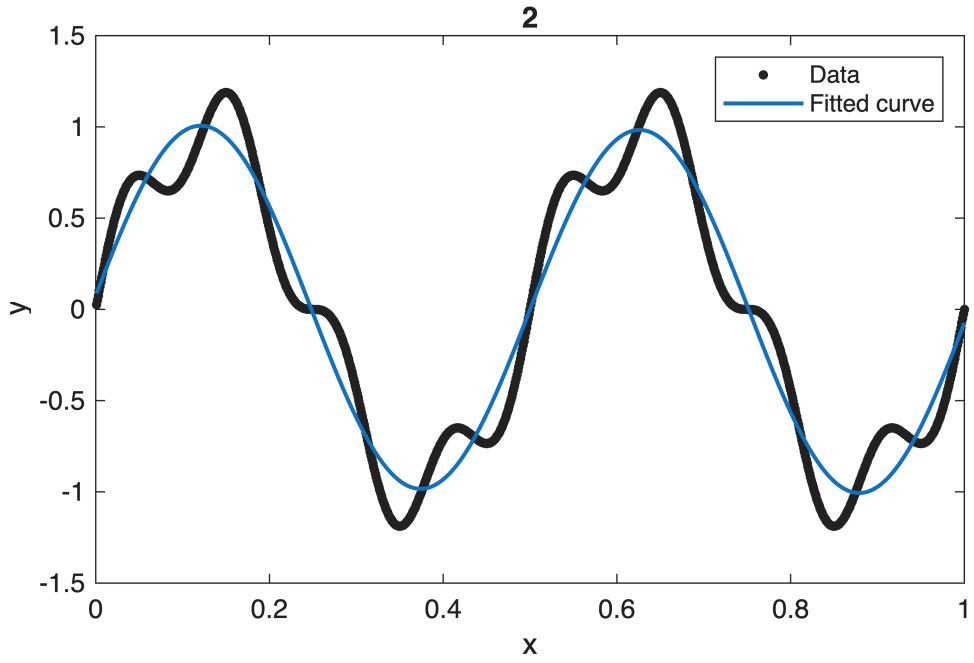
sin_N_harmonic = 1*sin(T*4*pi) + 0.25*sin(T*4*4*pi);

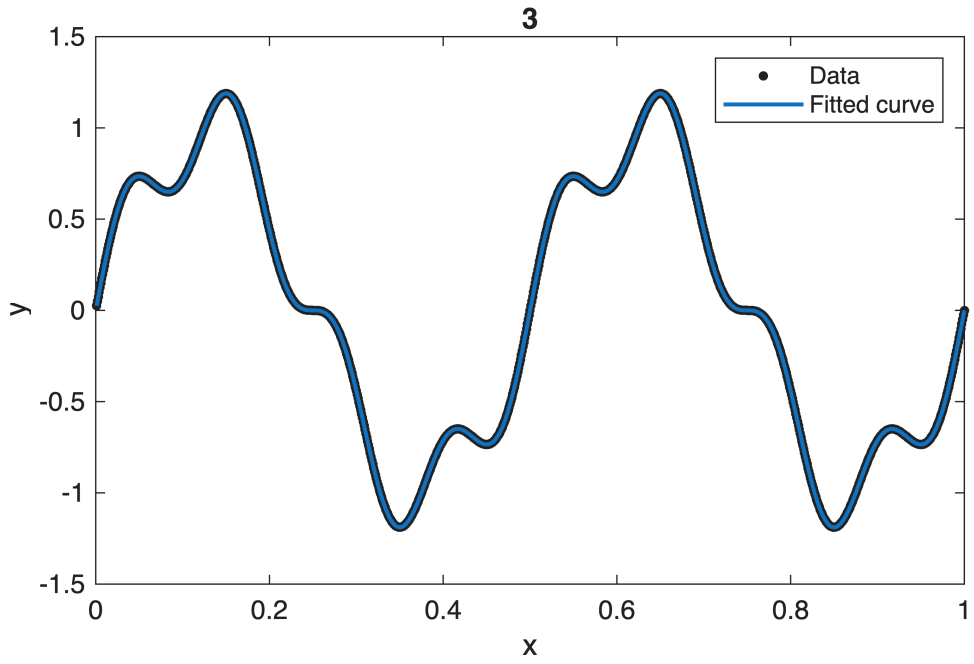
for i = 1:length(Nns)
    modelName = sprintf('fourier%d', Nns(i)); %bättre än att skriva ut flera
gånger

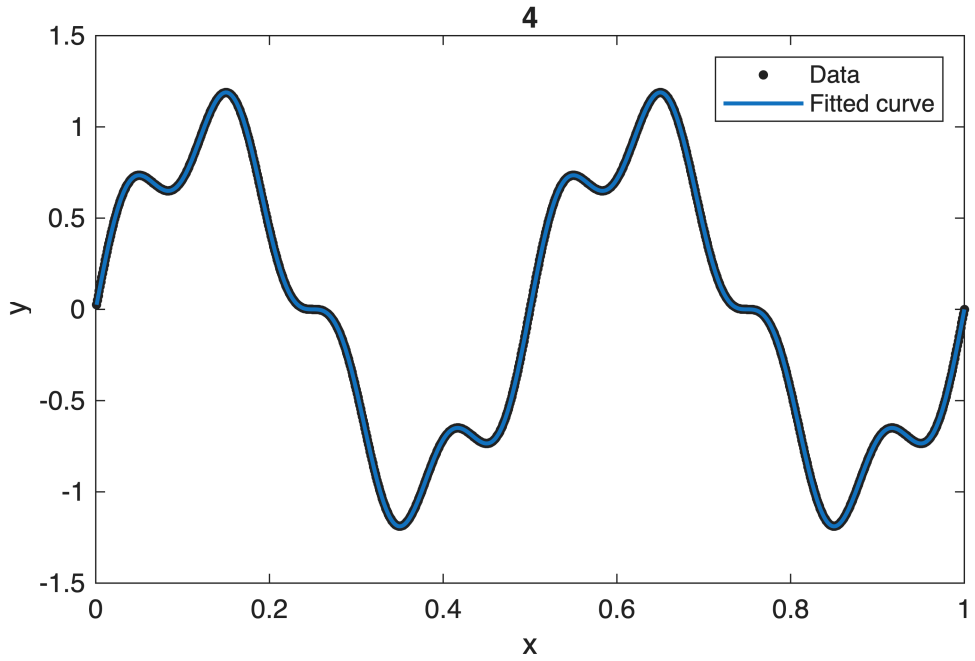
    [cN{i}, gN{i}] = fit(T, sin_N_harmonic, modelName);
    figure;
    plot(cN{i},T, sin_N_harmonic);
    title(i)

end
```




2
Data
Fitted curve
1.5
1
0.5
0
-0.5
-1
-1.5
y
0
0.2
0.4
0.6
0.8
1
x


3
Data
Fitted curve
1.5
1
0.5
0
-0.5
-1
-1.5
y
0
0.2
0.4
0.6
0.8
1
x



ANSWER: The coefficients are not precise enough on fourier1 and fourier2 but almost perfect on this time interval for fourier4 and fourier8

1.1.2. Fourier series of a sound signal

Calculate the Fourier series for one of the sound signals S1–S4 (same as used in **Task 1.2** of **Home Assignment 1**) using the command "`fit`" and Fourier series coefficients `N = 1, 2, 4 & 8` see example below. Plot the results and compare to the original signal.

```
[cN,gN] = fit(time_SX, SX, 'fourierN');
```

```
% add your code here
[SX, Fs ] = audioread("Sound1.wav");
Nns = [1,2,4,8];

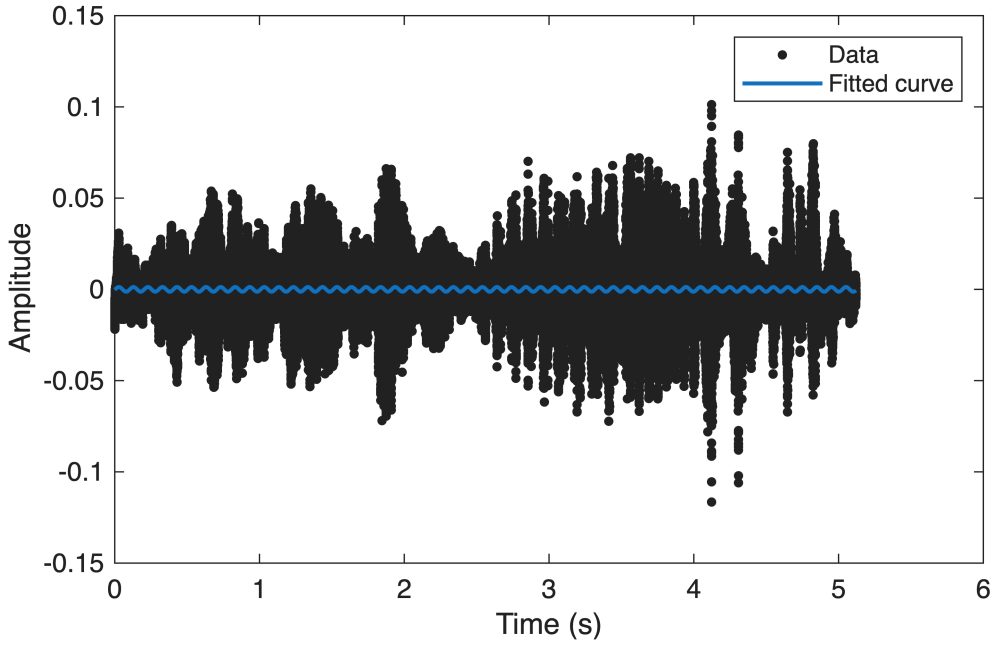
time_SX = (0:length(SX)-1)'/Fs;
SX = mean(SX, 2);

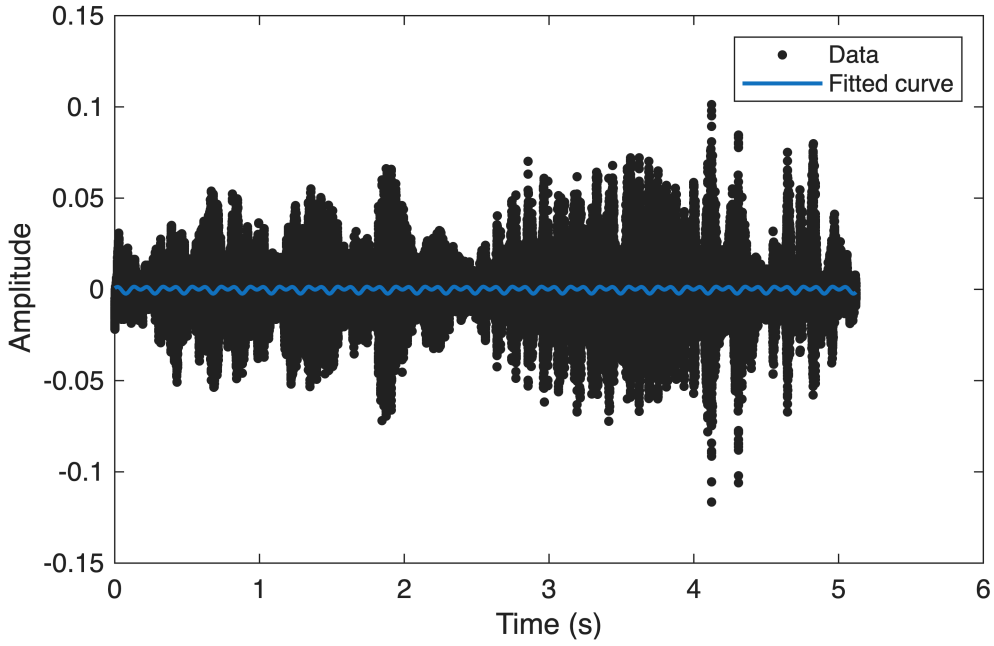
cN_SX = cell(1, length(Nns));
gN_SX = cell(1, length(Nns));

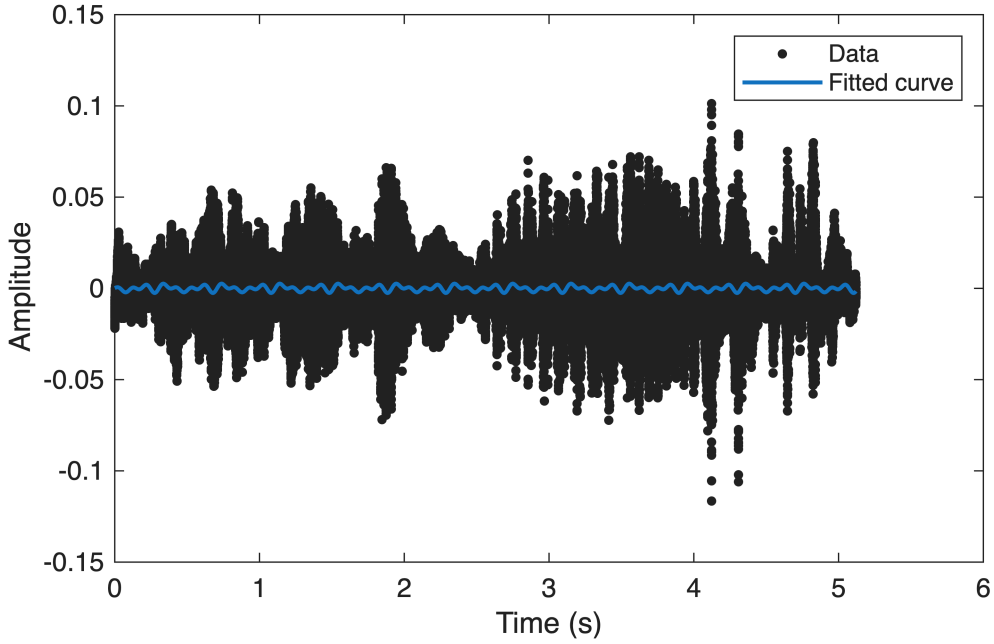
for i = 1:length(Nns)
    modelName = sprintf('fourier%d', Nns(i)); %bättre än att skriva ut flera
gånger

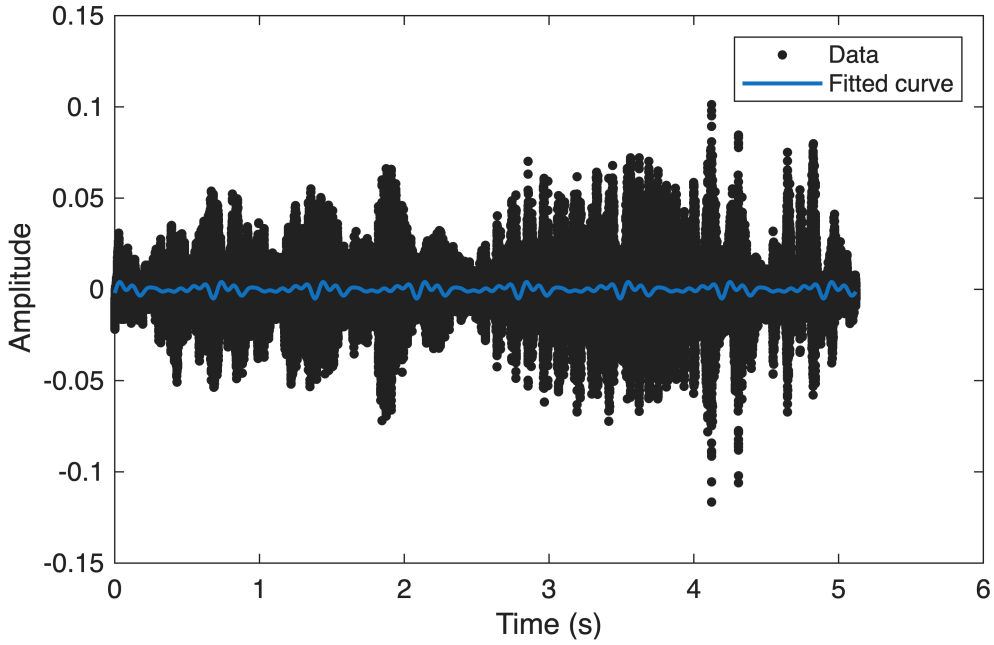
    [cN_SX{i}, gN_SX{i}] = fit(time_SX, SX, modelName);
    figure;
    plot(cN_SX{i},time_SX, SX);
    xlabel('Time (s)');
    ylabel('Amplitude');
```

```
end
```


Data
Fitted curve
Amplitude
Time (s)
0.15
0.1
0.05
0
-0.05
-0.1
-0.15
0
1
2
3
4
5
6


Data
Fitted curve
Amplitude
Time (s)
0.15
0.1
0.05
0
-0.05
-0.1
-0.15
0
1
2
3
4
5
6





ANSWER: For sound1 it cant do a fourier transform to make it match the data.

1.1.3. Explain the differences in the results seen in Tasks 1.1.1. and 1.1.2.

ANSWER: The result in task 1.1.1 is much better since the data is periodic, but in 1.1.2 a nonperiodic signal sound was chosen and therefore the result was quite lousy.

1.2 Fourier transform and inverse Fourier transform

Discuss with your colleague what kind of operation a Fourier transform is and what kind of output will be produced when a time series is Fourier transformed. Read carefully the MATLAB help on "`fft`" and then reread it one more time.

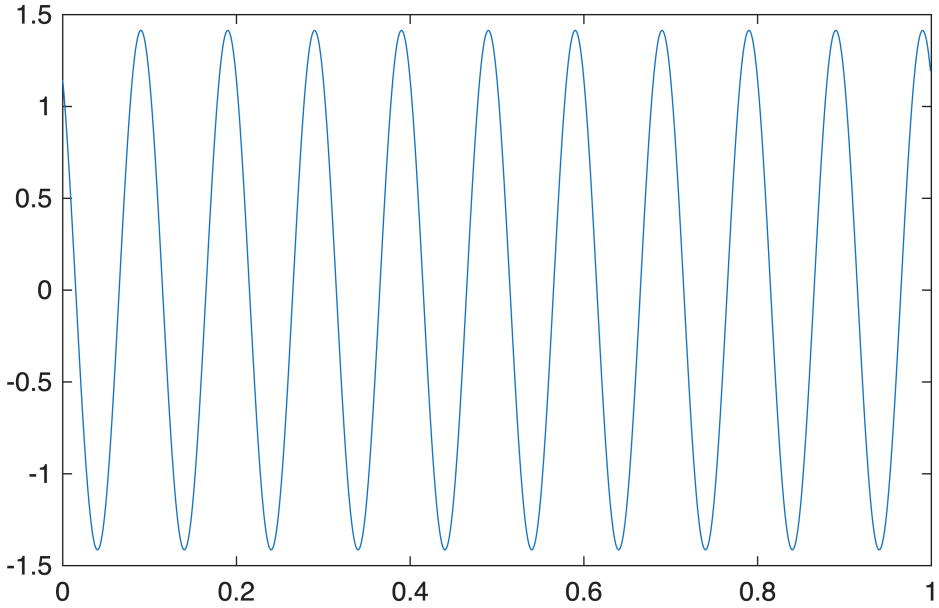
Now, create a simple sinusiodal with an added phase that is the the average of your ages divided by 10. Use the code:

```
freq=10;                                % frequency of stimuli (Hz)
fs = 1024;                              % Sample frequency (Hz)
t = 0:1/fs:1-1/fs;                      % 1 sec sample
A = sqrt(2);
x = A*sin(2*pi*freq*t+mean_age/10);
m = length(x);                          % Window length
N = pow2(nextpow2(m));                  % Transform length
y = fft(x,N);                           % DFT
```

```
% add your code here
freq=10;                                % frequency of stimuli (Hz)
fs = 1024;                              % Sample frequency (Hz)
t = 0:1/fs:1-1/fs;                      % 1 sec sample
A = sqrt(2);
mean_age = 22;
x = A*sin(2*pi*freq*t+mean_age/10);
m = length(x);                          % Window length
N = pow2(nextpow2(m));                  % Transform length
Y = fft(x,N);                           % DFT
```

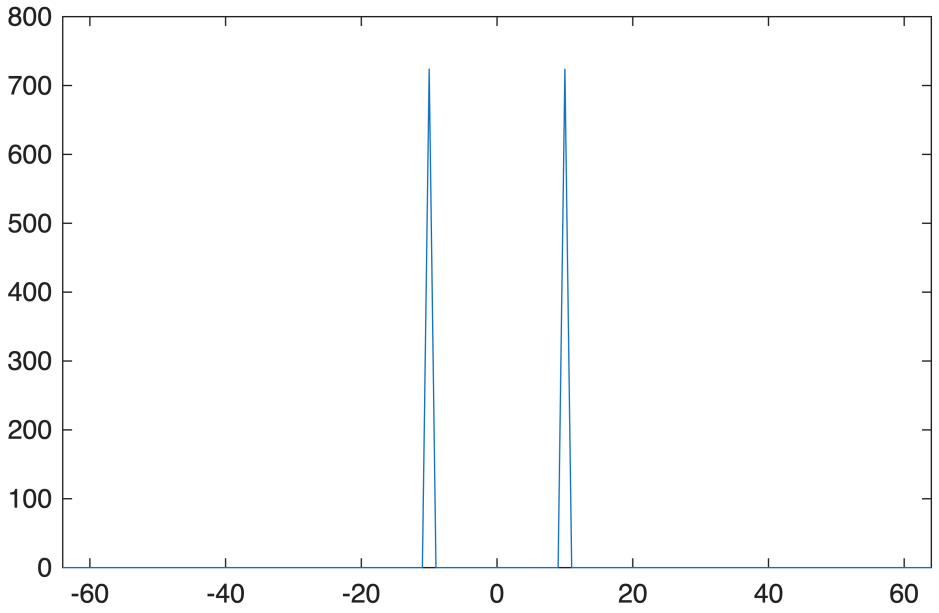
1.2.1. Plot the time signal x with the correct x- and y-axis

```
% add your plot here
plot(t, x)
```



1.2.2. Plot the raw and scaled output vector of the FFT magnitude spectrum

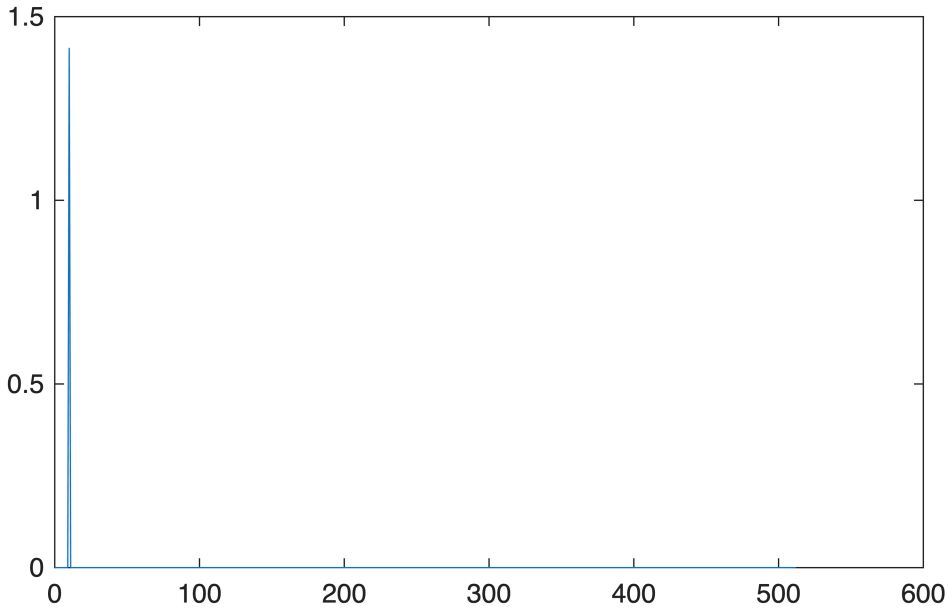
```
% add your plot here
Y2 = fftshift(Y);
z = (-N/2:N/2-1)/N * fs;
plot(z,abs(Y2))
xlim([-64,64])
```




```
%plot((0:N-1)*fs/N, real(Y))
```

The above corresponds to the raw amplitude spectrum of the signal. Proceed to scale the amplitude correctly in order to get a correct representation and plot it.

```
% add your plot here
P2 = abs(Y/N);
P1 = P2(1:N/2+1);
P1(2:end-1) = 2*P1(2:end-1);
f = fs/N*(0:(N/2));
plot(f, P1); %danke schön matlab help
```



1.2.3. Describe how the output vector y from `fft` is sorted

What frequency does the output vector's first element/index (n = 1) correspond to, and what does the last element (n = NFFT)? **Note that the MATLAB help can be unclear on this issue, so an additional check on the web is recommended.**

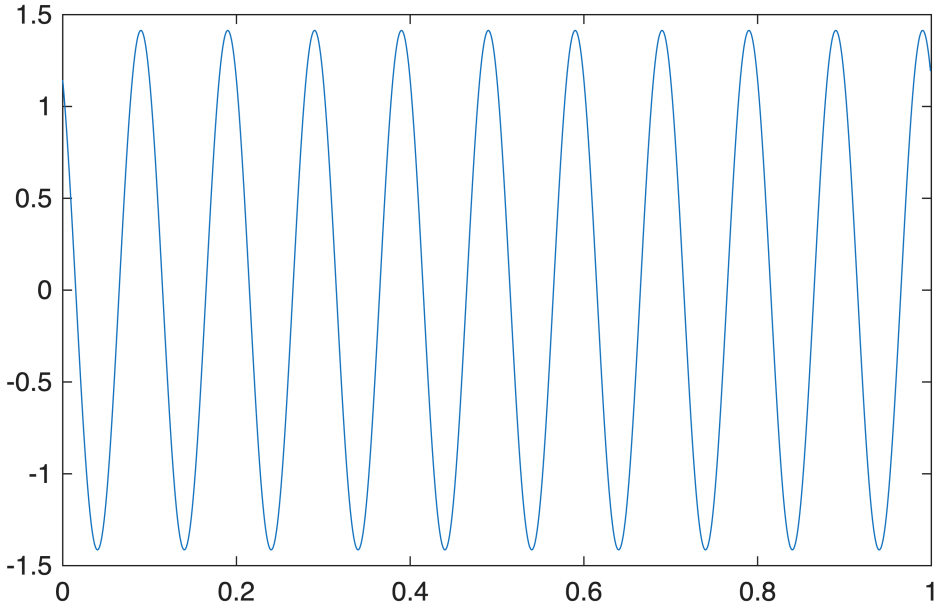
ANSWER: The output vector y from fft is sorted like this: index 1 is 0, first half is from 0 linearly up to the nyquist frequency (half of the sampling frequency). second half is the same but from negative nyquist frequency to just up to 0. An fftshift-command will make 0 sorted to the middle.

1.2.4. Verify the inverse FFT of a forward FFT

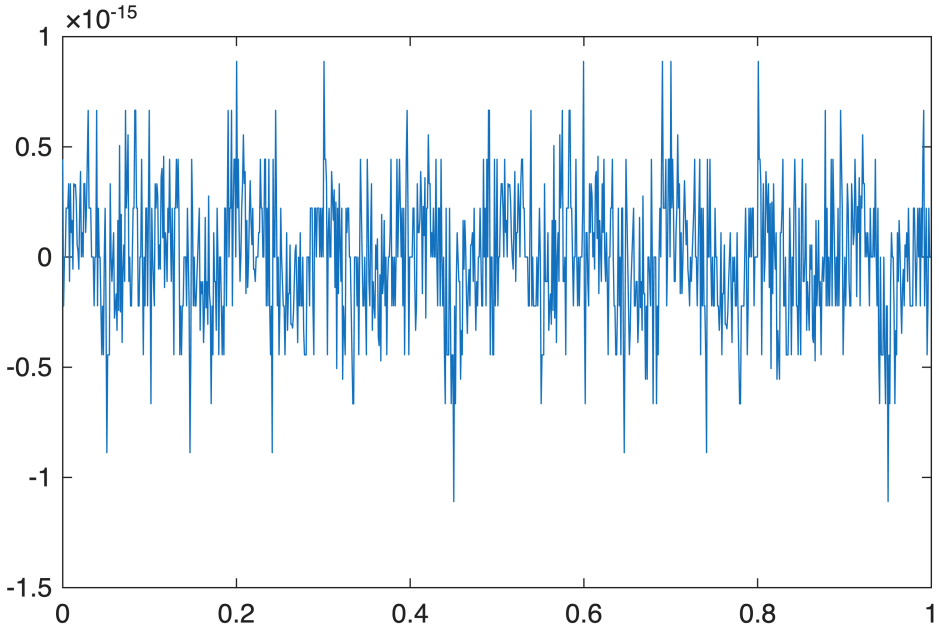
Use the inverse Fourier transform command "`ifft`" on the output from the "`fft`" results from the signal above

```
% add your code here
x_inverseY = ifft(Y, N);

plot(t, x_inverseY)
```




```
plot(t,x_inverseY-x) % ger väldigt litet fel
```

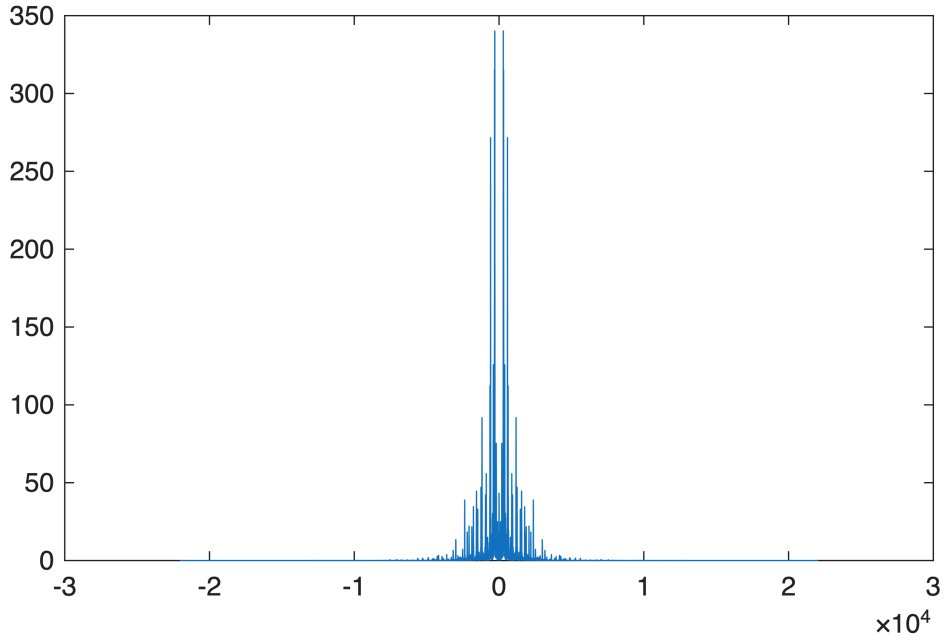


Is the result equal to the original time signal?

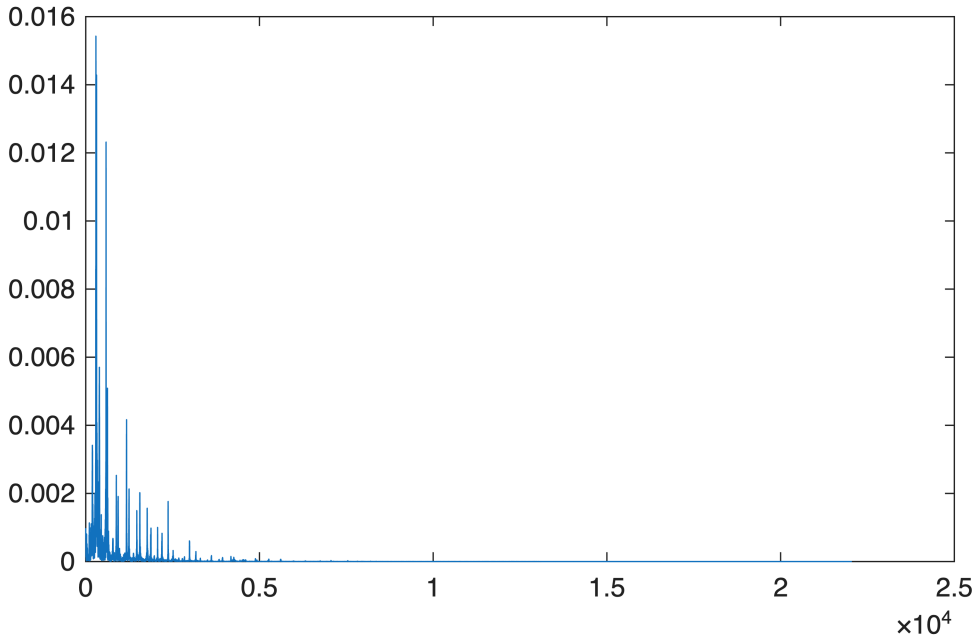
ANSWER: Yes

1.2.5. Read and FFT the file "*Classic.wav*" with the code you used in Task 1.2.2, set `n = 1024`

```
% add your code here
[y_cl, Fs_cl] = audioread("Classic.wav"); %y_cl är 352800 datapunkter stor
n = 1024;
Y_cl = fft(y_cl,n );
Y_cl= fftshift(Y_cl);
Z = (-n/2:n/2-1)/n * Fs_cl;
plot(Z,abs(Y_cl))
```




```
Y_cl = ifftshift(Y_cl);
P2 = abs(Y_cl/n);
P1 = P2(1:n/2+1);
P1(2:end-1) = 2* P1(2:end-1);
f = Fs_cl/n*(0:(n/2));
plot(f, P1);
```



How large part of the original sound is used? What has been analyzed in the remaining signal? Comment on the consequences on bandwidth, computational time, and RAM usage if you were to determine the frequency content of a 4-minute song with a sampling frequency of 44100 Hz?

ANSWERS: approx 0.023 s were used. 1024/44100 = 0.023 or number of samples / sampling freq = time. In the remaining signal nothing has been analyzed.

A 4-min long song with Fs = 44,1 kHz would be N = 240 * 44100 which is ca 10,5 million samples.

Consequences of bandwidth: deltaF = Fs / N which would be very good and small. Very precise. The FFT does analyse the whole signal at once though so the time-domain information would be gone.

Computational time: FFT has log linear time complexity so it would become very big very fast. Analyzing the song in bits of 1024 samples is faster.

RAM usage would also be very big since its 10,5 million samples.

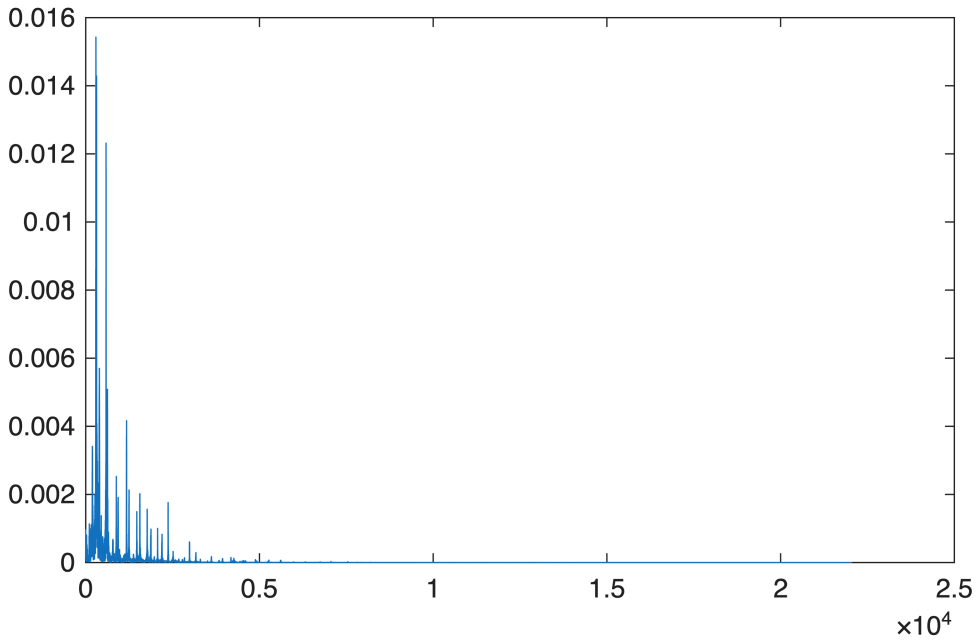
2: SAMPLING

Advice: When plotting discrete spectra, the command "`stem`" is better than "`plot`" to see the properties of the different frequencies. Therefore, use the former command when appropriate in the exercises below. OBS: **Tasks 2.1** & **2.2** here are similar to **Tasks 3.1.3** & **3.2.3** in Home Assignment 1, so copy and paste.

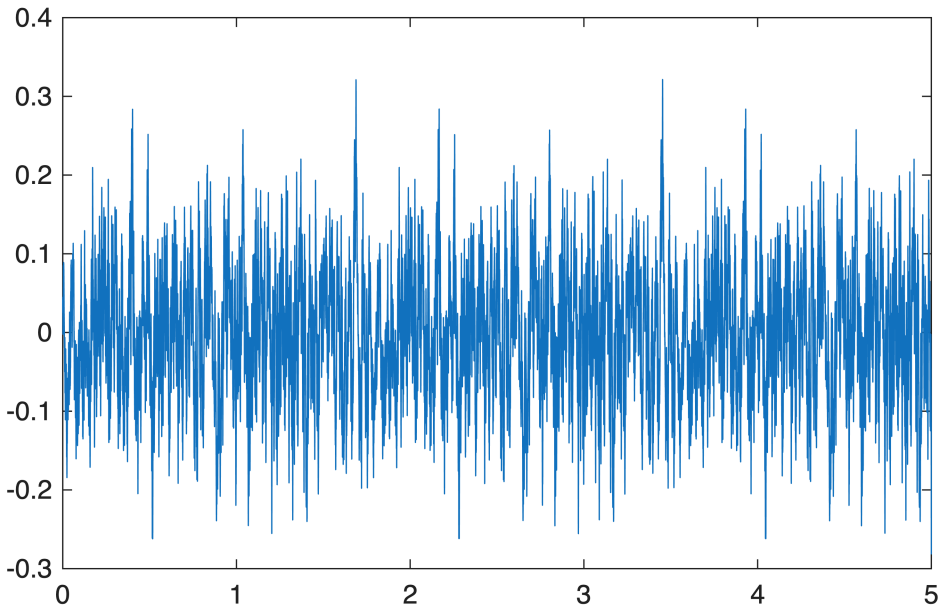
2.1. Read the Sound4.wav 4 s sample using "`audioread`"

Plot the data in the time domain and listen to it using “audioplayer”.

```
% add your code here
[Y,Fs] = audioread("Sound4.wav");
Y = (Y(:,1)+Y(:,2))/2;
t = (0:length(Y)-1)'/Fs;
hold on;
```




```
figure;
plot(t,Y)
```



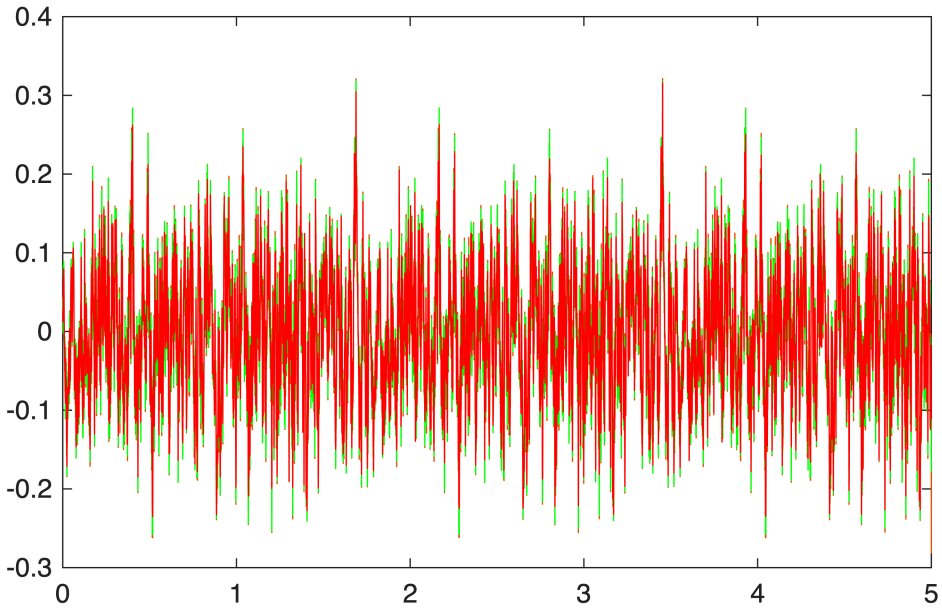
```
sound(Y,Fs);
```

2.2. Resample the signal at every 4th and 32nd points

Plot the resampled signals in the same graph as the original sample using "`resample`" in MATLAB.

```
% add your code here

Yrs4 = resample(Y,1,4);
Yrs32 = resample(Y,1,32);
hold on
plot(t, Y);
plot(t(1:4:end),Yrs4,'g');
plot(t(1:32:end),Yrs32,'r');
```




```
sound(Yrs32,Fs/32);
```

Describe the difference between the three curves by zooming in on a small time segment.

ANSWER: The difference is as expected, the original sample has the most data points/ the best resolution. whilst as intended the sampling of every 4th point still manages to follow the original closely but the 32 point sampling does flaten out alot of detail

2.3. Plot one-sided spectra using 512 samples for all three sampling frequencies (Fs, Fs/4, Fs/32) and compute the frequency resolution (Δf) for all these spectra

```
% add your code here
N = 512;

Fs4 = Fs/4;
Fs32 = Fs/32;

df = Fs/N;
df4 = Fs4/N;
df32 = Fs32/N;

fprintf('delta_f (Fs):  %.4f Hz\n',df)
```

```
delta_f (Fs):  62.5000 Hz
```

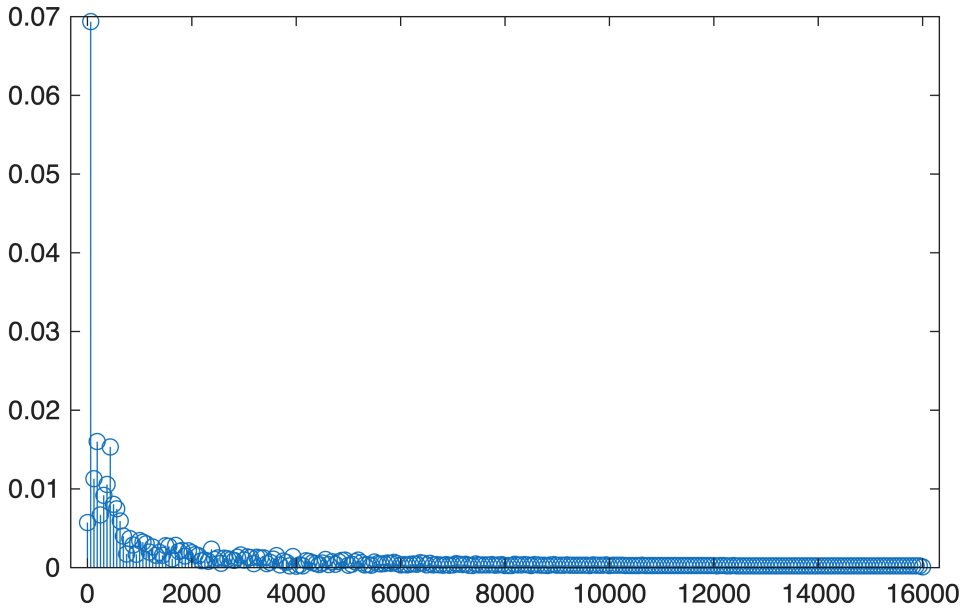
```
fprintf('delta_f (Fs):  %.4f Hz\n',df4)
```

```
delta_f (Fs):  15.6250 Hz
```

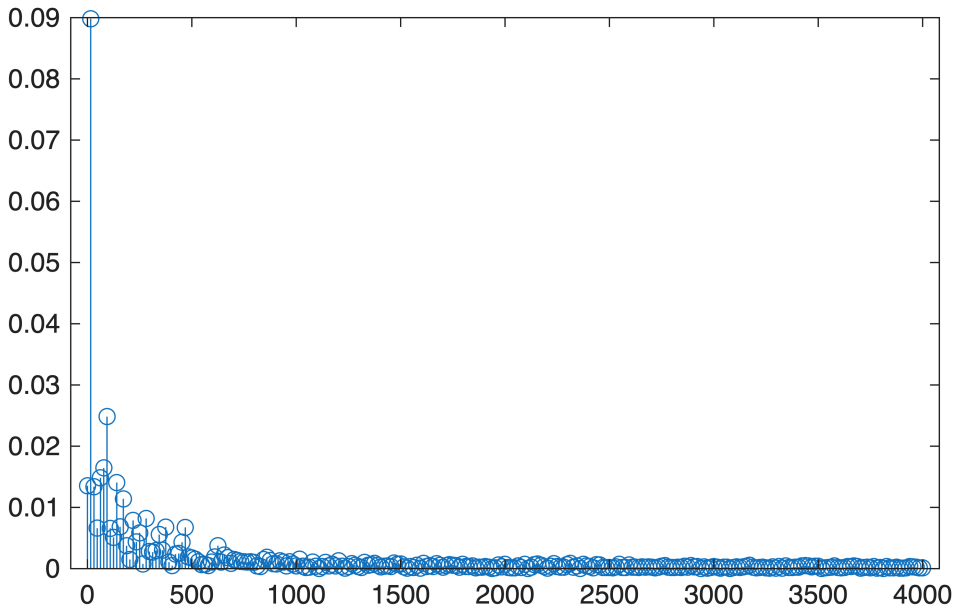
```
fprintf('delta_f (Fs):  %.4f Hz\n',df32)
```

```
delta_f (Fs):  1.9531 Hz
```

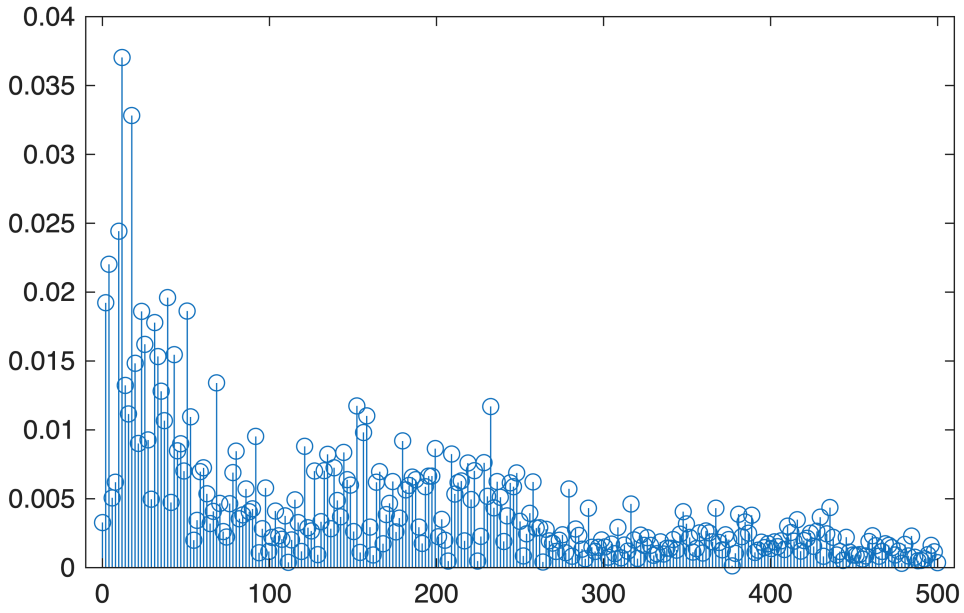
```
fft1 = fft(Y(1:N),N)/N;
spektra = abs(fft1(1:N/2+1)); % tar fram ensidiga spektrat
spektra(2:end-1) = 2*spektra(2:end-1); % corrects energies
f = (0:N/2)*df;
fft4 = fft(Yrs4(1:N),N)/N;
spektra4 = abs(fft4(1:N/2+1)); % tar fram ensidiga spektrat
spektra4(2:end-1) = 2*spektra4(2:end-1); % corrects energies
f4 = (0:N/2)*df4;
fft32 = fft(Yrs32(1:N),N)/ N;
spektra32 = abs(fft32(1:N/2+1)); % tar fram ensidiga spektrat
spektra32(2:end-1) = 2*spektra32(2:end-1); % corrects energies
f32 = (0:N/2)*df32;
figure;
stem(f,spektra);
```




```
figure;
stem(f4,spektra4);
```




```
figure;
stem(f32,spektra32);
```



Explain how the amplitudes differ for each sampled signal and explain why that is so.

ANSWERS: *The highest amplitude exists in the sample of every fourth point, the original has a slightly lower max whilst the 32 sampling is suprisingly lower. Might be because the average for that frequency is lower over a longer time period. furthermore more detail is picked up from the more downsampled since df is much lower, but it only covers up to 500 Hz.*

2.4. Listen to the downsampled signals and compare them to the original recording

Remember to change the sampling frequency in the "`audioplayer`" input.

```
% add your code here
sound(Y,Fs)
pause(5)
sound(Yrs4,Fs/4);
pause(5)
sound(Yrs32,Fs/32);
```

Describe the differences.

ANSWERS: *The difference between the original and the one resampled at every foruth point is small, noticable but small. the resample of every 32 points however really dampens the noise. in the first two you can hear details, in the last one you only hear the background.*

3: ALIASES

Reuse the 10 Hz sinusoid from Task 1.2.

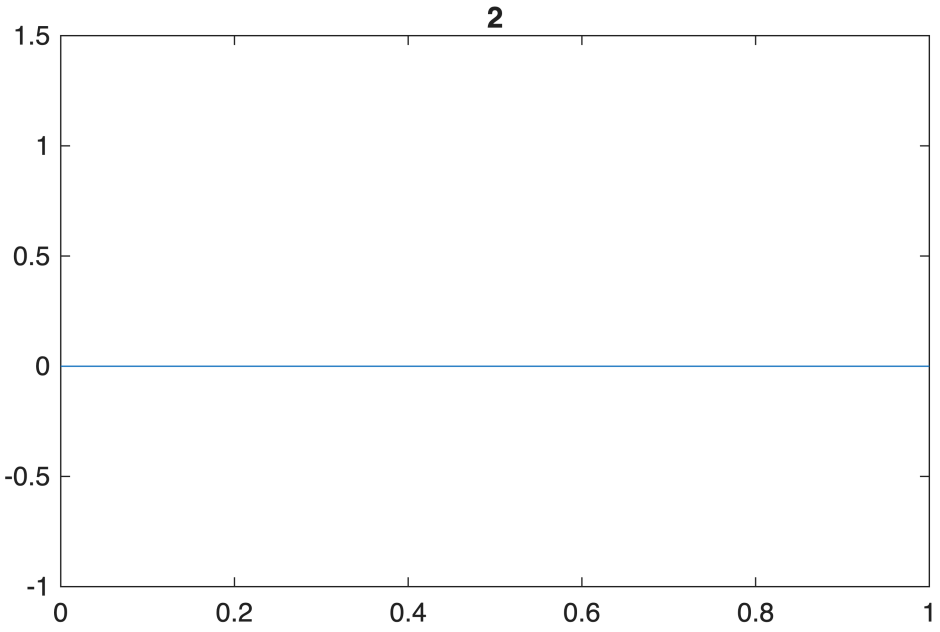
```
freq = 10;                              % frequency of stimuli (Hz)
fs = 1024;                              % Sample frequency (Hz)
t = 0:1/fs:1-1/fs;                      % 1 sec sample
x = sqrt(2)*sin(2*pi*freq*t);
m = length(x);                          % Window length
N = pow2(nextpow2(m));                  % Transform length
y = fft(x,N);                           % DFT
```

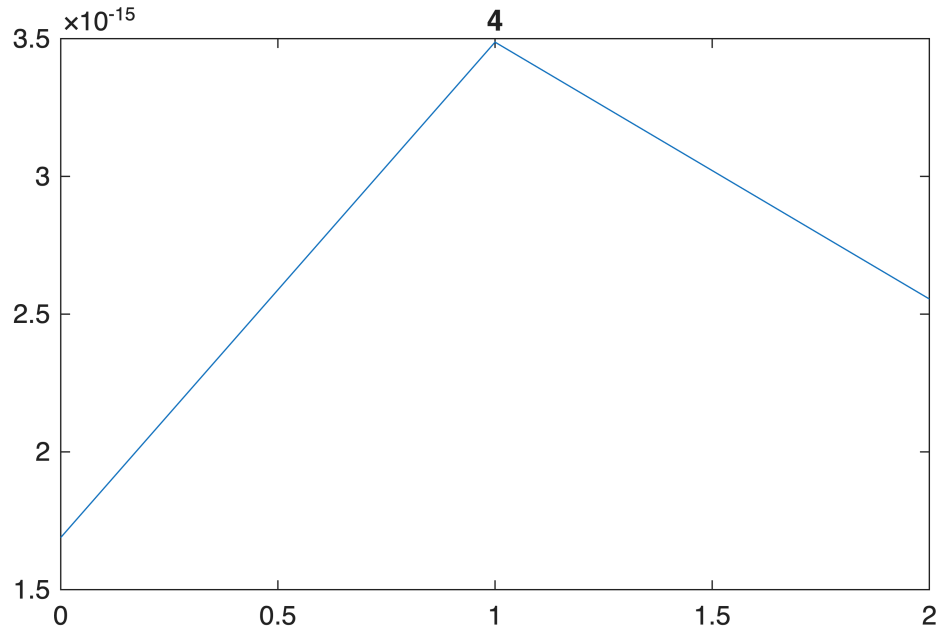
3.1. Sample the sinusoid with 32 Hz, 20 Hz, 16 Hz, and 8 Hz

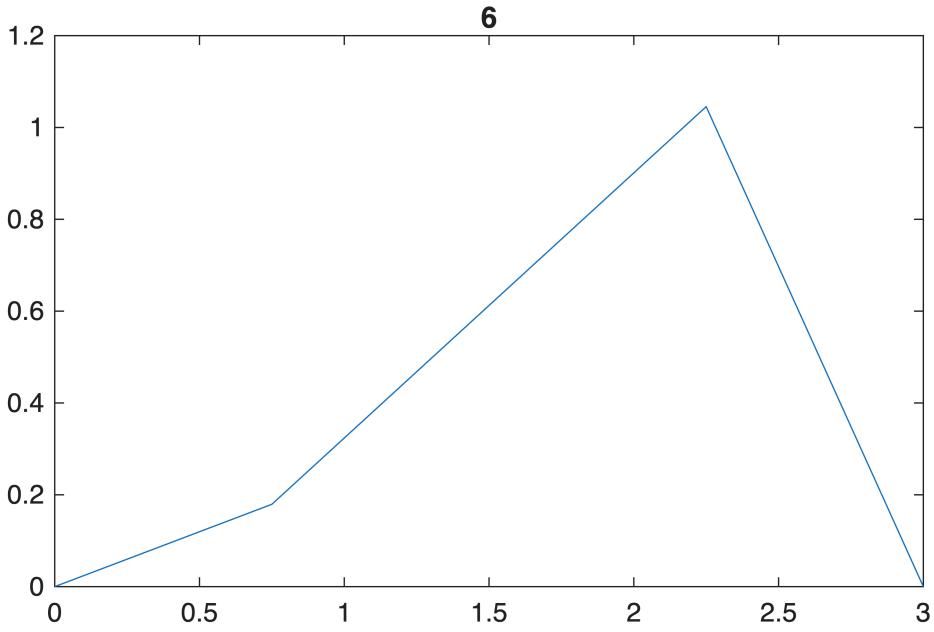
```
% add your code here
freq = 10;                                % frequency of stimuli (Hz)
samplingfrequencies = 2:2:32;

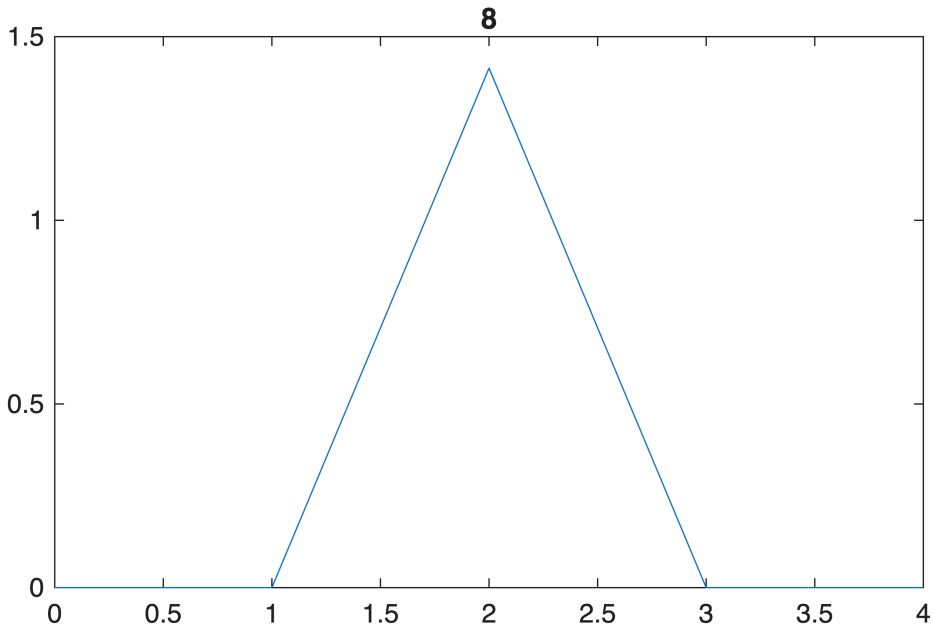
for j = 1:length(samplingfrequencies)  % Sample frequency (Hz)
    fs = samplingfrequencies(j);
t = 0:1/fs:1-1/fs;                        % 1 sec
sample
x = sqrt(2)*sin(2*pi*freq*t);
m = length(x);                            % Window length
N = pow2(nextpow2(m));                    % Transform length
y = fft(x,N);

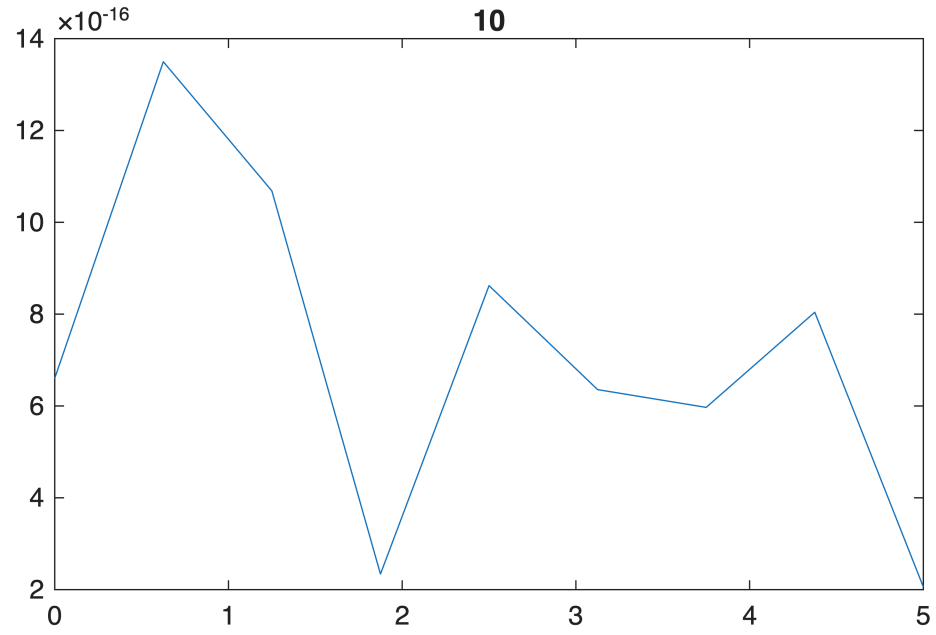
y = abs(y/N);
y = y(1:N/2+1);
y(2:end-1) = 2*y(2:end-1);
f = fs/N*(0:(N/2));
figure;
plot(f, y);
title(samplingfrequencies(j))
end
```

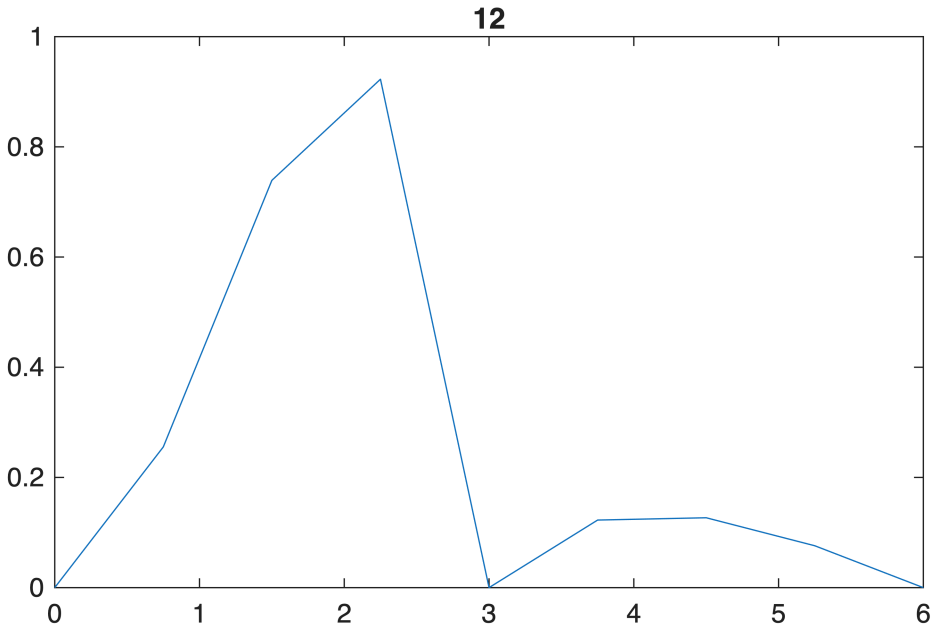

2
1.5
1
0.5
0
-0.5
-1
0
0.2
0.4
0.6
0.8
1

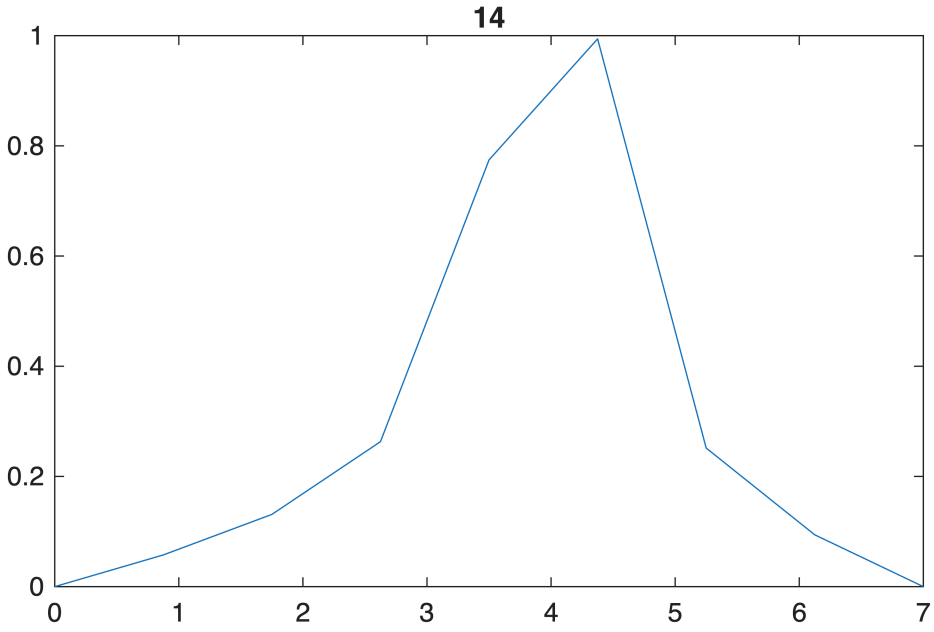

4
×10-15
3.5
3
2.5
2
1.5
0
0.5
1
1.5
2

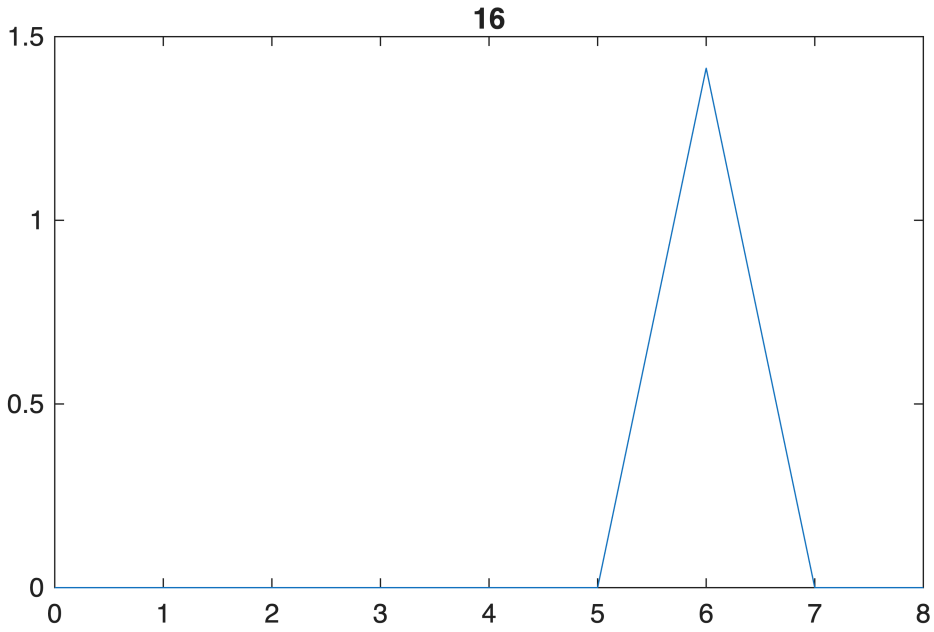

6
1.2
1
0.8
0.6
0.4
0.2
0
0
0.5
1
1.5
2
2.5
3

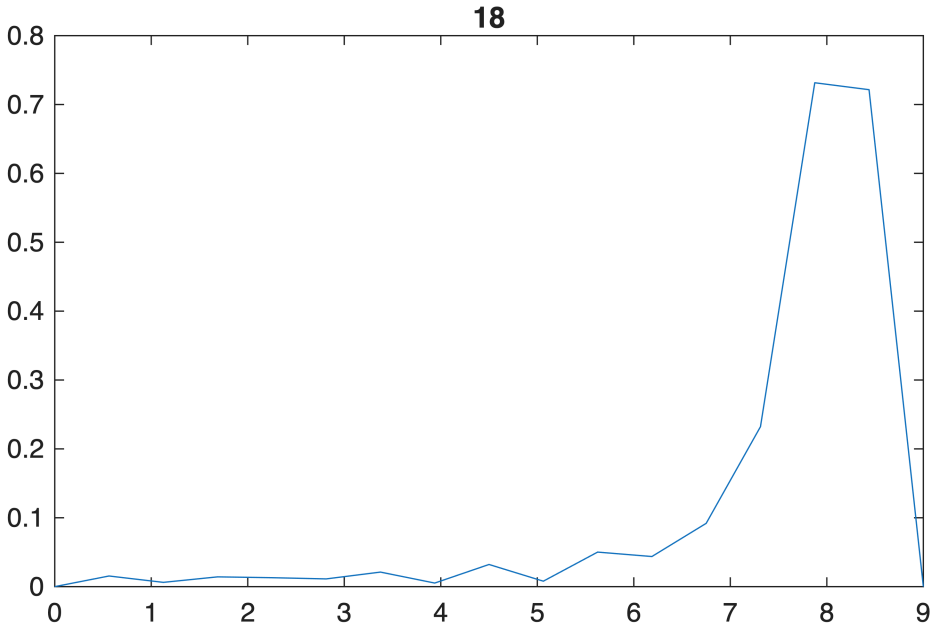

8
1.5
1
0.5
0
0
0.5
1
1.5
2
2.5
3
3.5
4

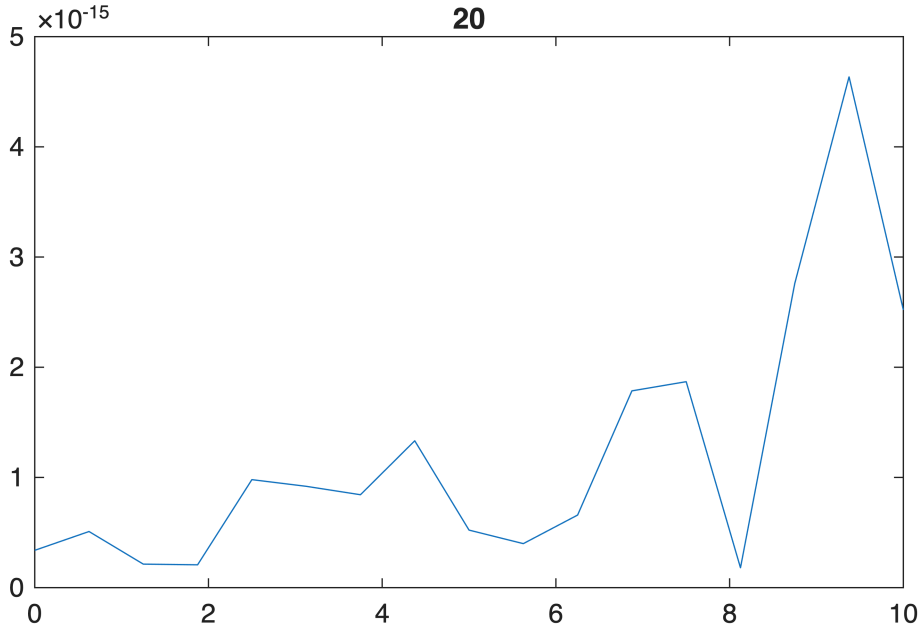

10
×10-16
14
12
10
8
6
4
2
0
1
2
3
4
5

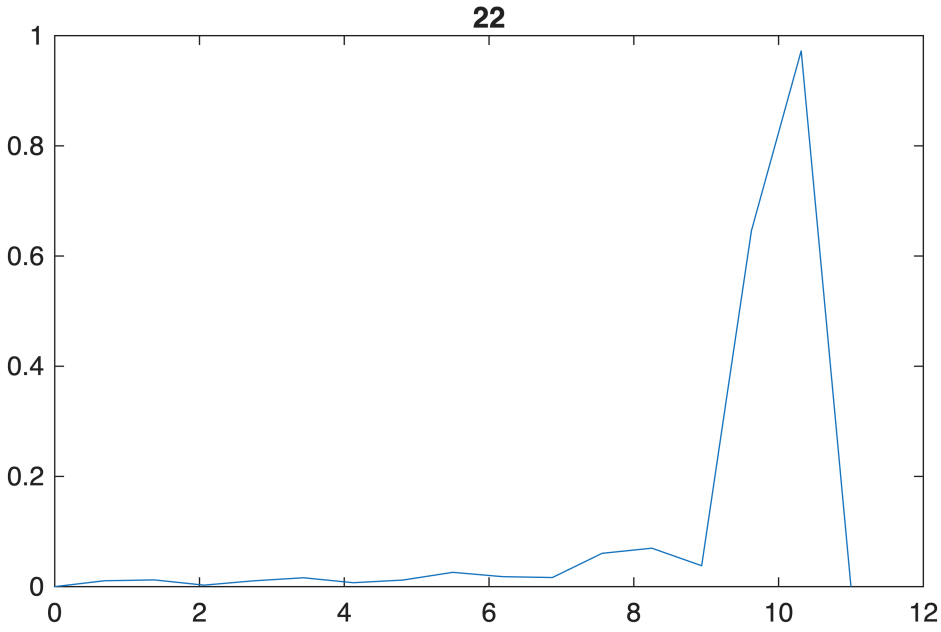

12
1
0.8
0.6
0.4
0.2
0
0
1
2
3
4
5
6

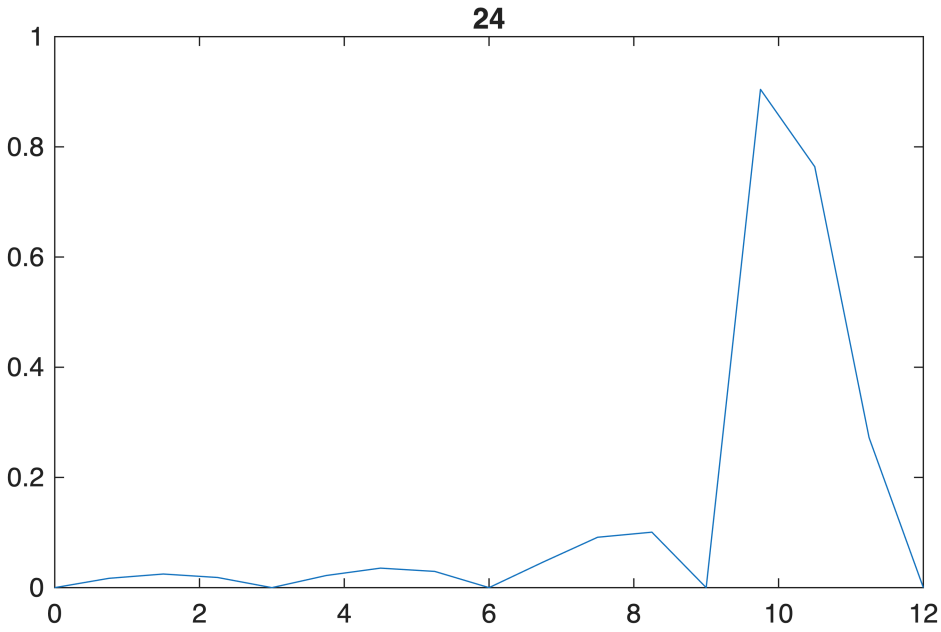

14
1
0.8
0.6
0.4
0.2
0
0
1
2
3
4
5
6
7

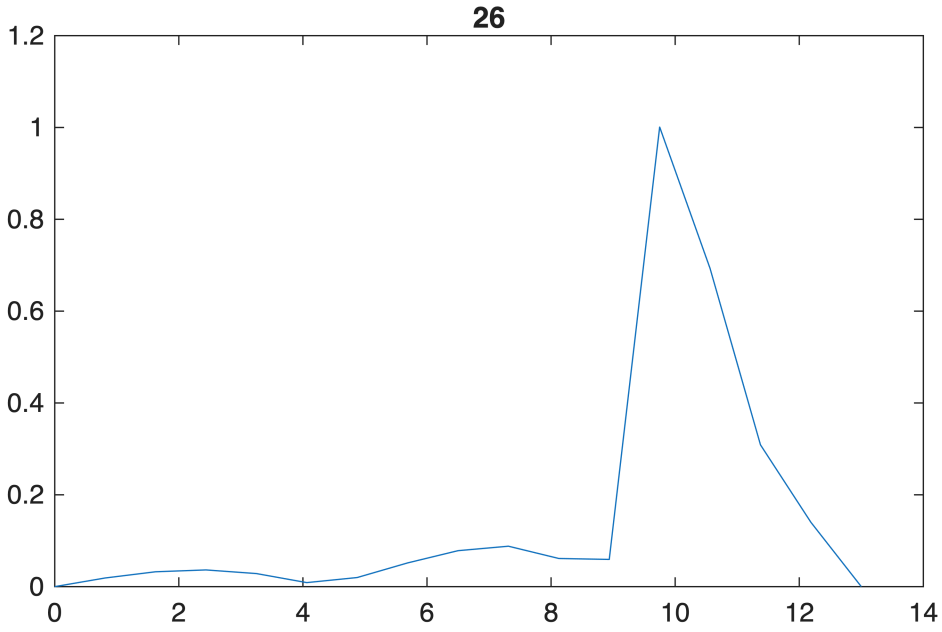

16
1.5
1
0.5
0
0
1
2
3
4
5
6
7
8

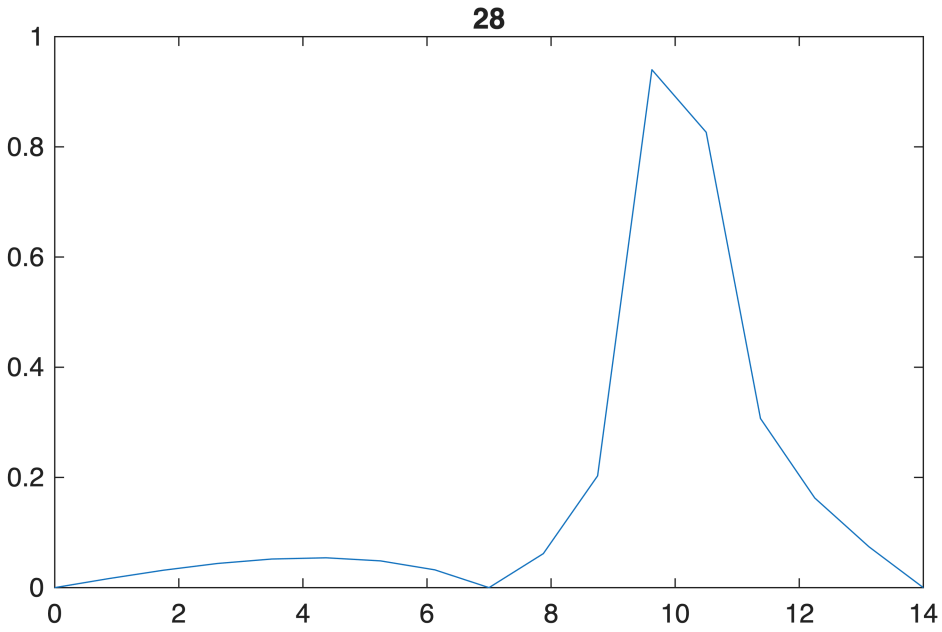

18
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0
1
2
3
4
5
6
7
8
9

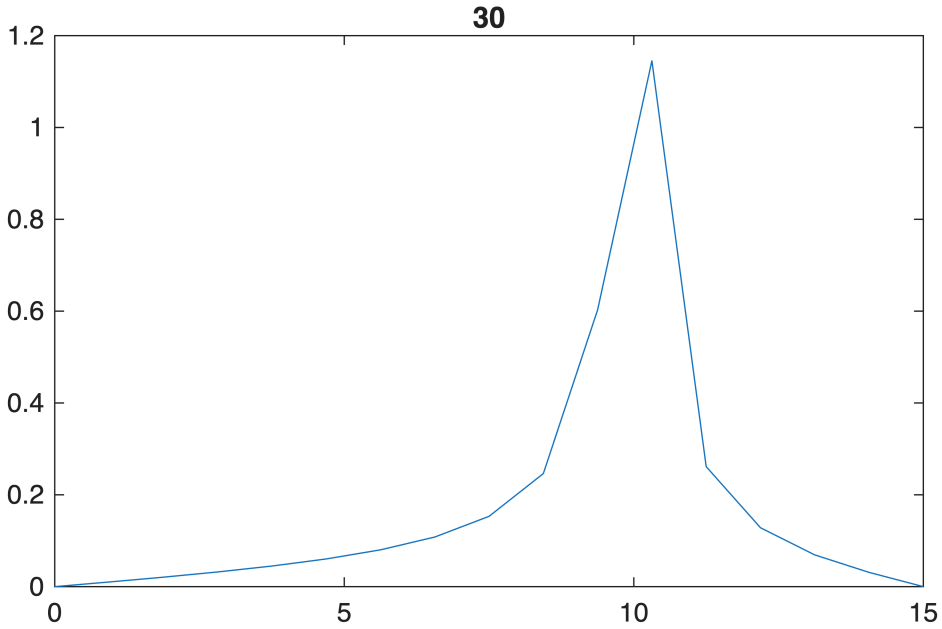

×10-15
20
5
4
3
2
1
0
0
2
4
6
8
10

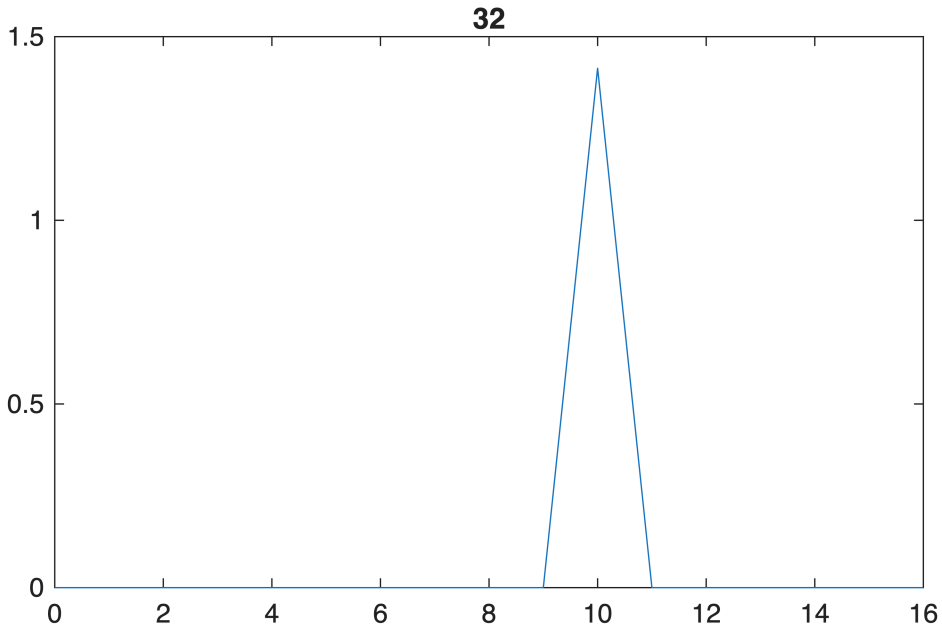

22
1
0.8
0.6
0.4
0.2
0
0
2
4
6
8
10
12


24
1
0.8
0.6
0.4
0.2
0
0
2
4
6
8
10
12


26
1.2
1
0.8
0.6
0.4
0.2
0
0
2
4
6
8
10
12
14


28
1
0.8
0.6
0.4
0.2
0
0
2
4
6
8
10
12
14





Comment on the resulting frequency plots. When can you observe aliasing?

ANSWERS: Aliasing can be observed on frequencies lower than the nyquist-freq.

4: SPECTROGRAM

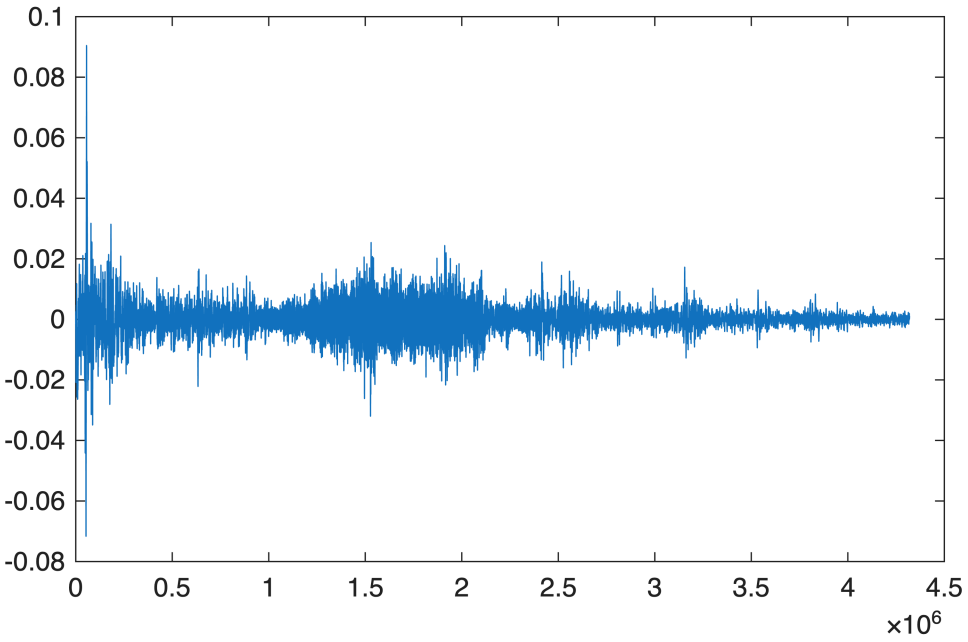
In this assignment, you are investigating a recording of an aircraft (Airbus A320) flyover and will estimate the speed of the plane from this recording “A320”. Start by listening to the recording. You should be able to hear a clear tonal sound during the recording, along with an eventually decreasing pitch (frequency of the tone). The

sound source is caused by cavities for fuel over pressure disposure situated under the wings and is known in aeroacoustics as a Helmholtz resonator.

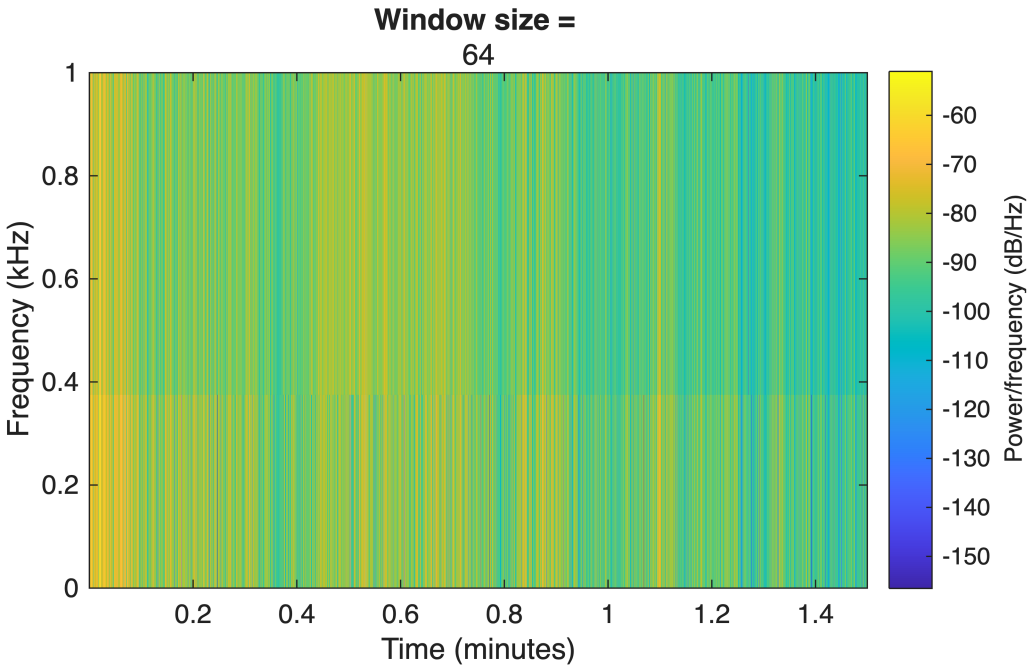
Now, use the spectrogram function "`spectrogram`" to analyze the signal in the time- and frequency domain simultaneously. Plot spectrograms with different window sizes, the first using 64 as window size and then increasing the window size. Hint: The tones have frequencies below 1 kHz so zoom in the low frequencies.

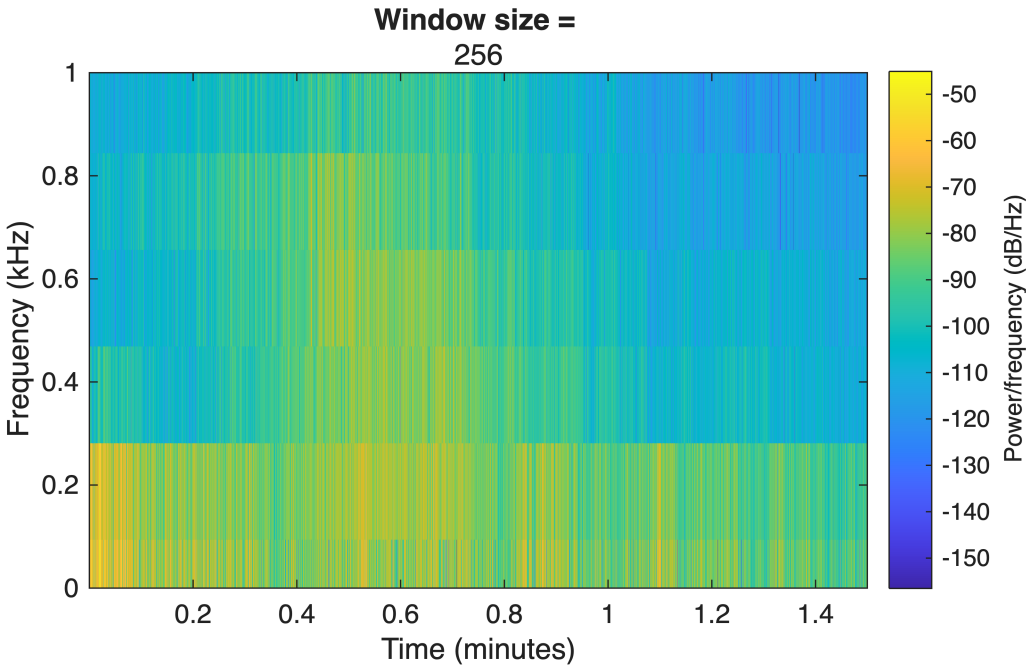
4.1. Plot the spectrogram for varying window size

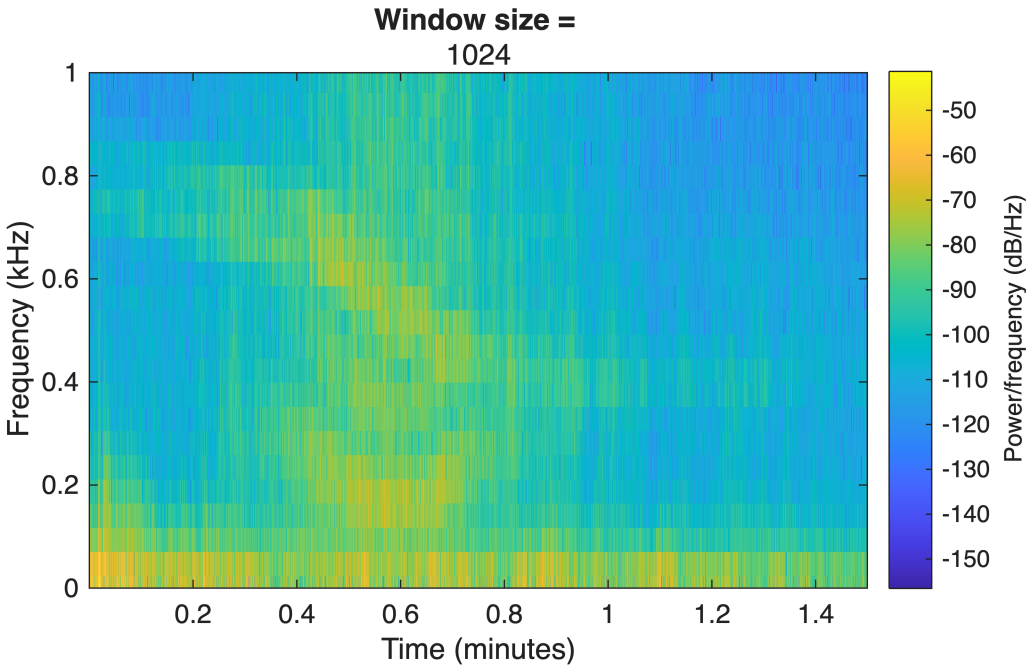
```
% add your code here
[Y,Fs] = audioread("A320_232.wav");
figure;
%sound(Y,Fs);
plot(Y)
```

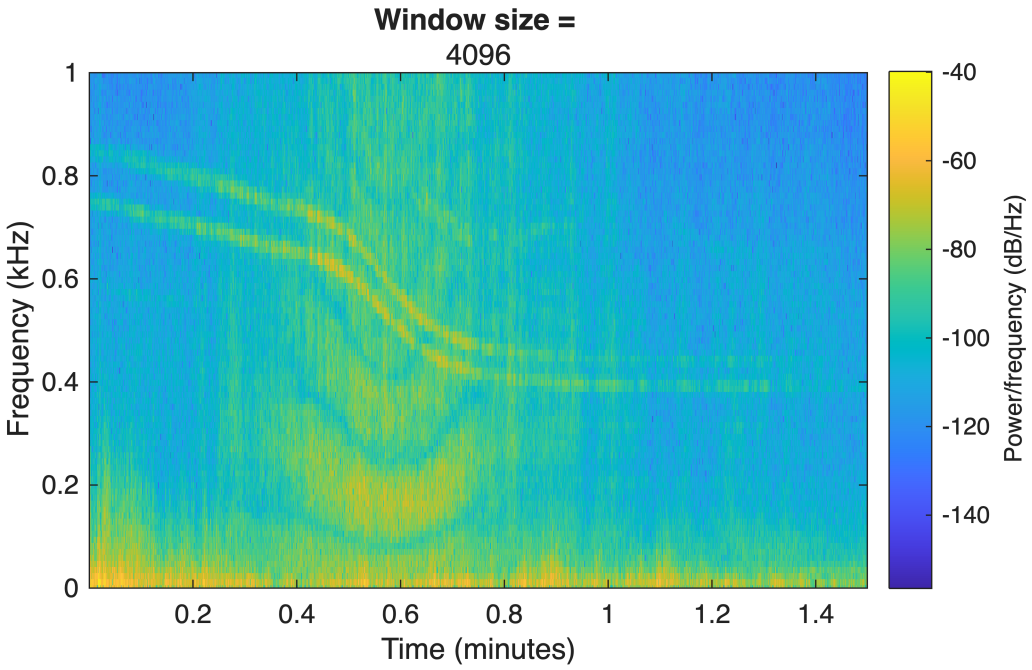


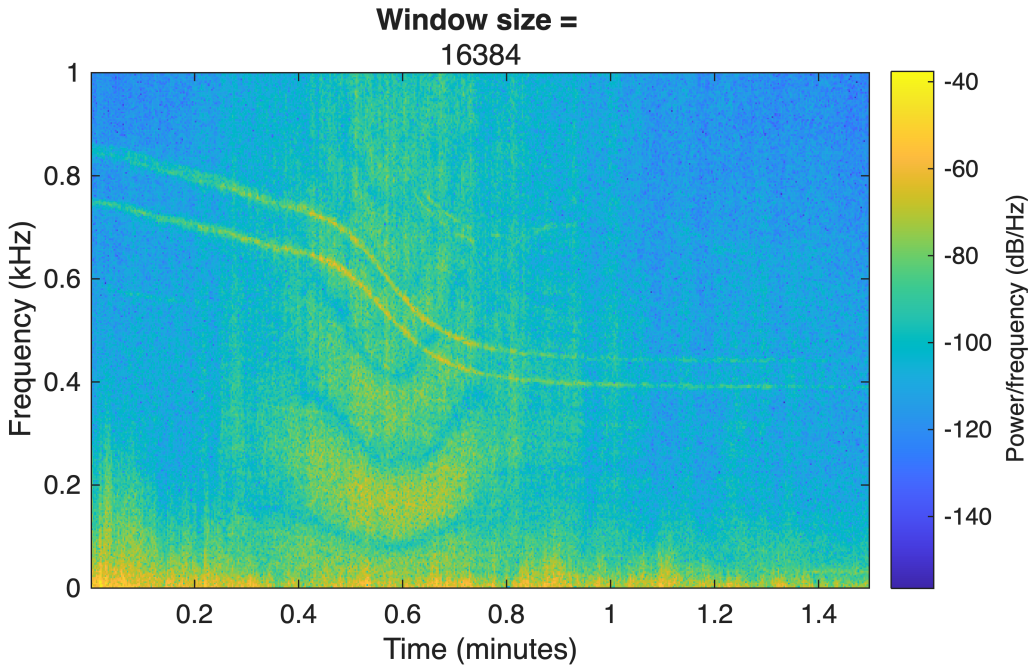

```
for i = 1:5
    figure;
    w = 2^(4+2*i); noverlap = w/2; freqSpec = w;
    spectrogram(Y,w,noverlap,freqSpec,Fs,'yaxis');
    ylim([0,1]);
    title('Window size = ', num2str(w));
end
```


Window size =
64
Frequency (kHz)
Time (minutes)
Power/frequency (dB/Hz)
0
0.2
0.4
0.6
0.8
1
0.2
0.4
0.6
0.8
1
1.2
1.4
-60
-70
-80
-90
-100
-110
-120
-130
-140
-150


Window size =
256
Frequency (kHz)
Time (minutes)
Power/frequency (dB/Hz)
0
0.2
0.4
0.6
0.8
1
0.2
0.4
0.6
0.8
1
1.2
1.4
-50
-60
-70
-80
-90
-100
-110
-120
-130
-140
-150


Window size =
1024
Frequency (kHz)
Time (minutes)
Power/frequency (dB/Hz)
0
0.2
0.4
0.6
0.8
1
0.2
0.4
0.6
0.8
1
1.2
1.4
-50
-60
-70
-80
-90
-100
-110
-120
-130
-140
-150


Window size =
4096
Frequency (kHz)
Time (minutes)
Power/frequency (dB/Hz)
0
0.2
0.4
0.6
0.8
1
0.2
0.4
0.6
0.8
1
1.2
1.4
-40
-60
-80
-100
-120
-140



How do the time and frequency domain resolutions interact?

ANSWER: *For the larger window sizes the resolution becomes much better and it is possible to se two clear lines that decrease in frequency over time. As we can hear in the sound clip. However if we zoom in we can see the trade of time resolution for frequency resolution as the window gets bigger.*

4.2. Zoom-in and examine the spectrogram

```
% add your code here
```

How many Helmholtz resonators are situated under the wing, i.e., how many different tones do you see in the spectrogram?

ANSWER: *<From the previous plots we can see two clear lines indicating that there are two helmholtz resonators on the aircraft, presumably one per wing going from 860Hz to 460Hz and from 750Hz to 400Hz. interesting however is that there seems to be a third very faint line going from 600Hz to about 350Hz.*

4.3. Use the Doppler shift in the spectrogram to estimate the aircraft's speed

The Doppler shift on the microphone measurement is estimated by

$$f' = f\frac{v}{v+v_m},$$

where f' is the observed frequency, f is the emitted frequency both you will need to estimate from the observed spectrogram, v is the speed of sound (in air), and v_m is the speed of the aircraft ($v_m < 0$ for the approach and $v_m > 0$ when the plane has passed the microphone). These formulas do not consider the aircraft is 700 m above the microphone. Thus, the relative speed of the aircraft can be estimated by dividing v_m by *sin(45°)* as this will approximately give the angle between the velocity arrow and microphone position.

ANSWER: *< We will assume that the speed is the same as the aircraft travels over the microphone. We can then use two observations, one for approach and one for passed. som f'1 = 750Hz, f'2 = 400Hz. since we now have a system of two equations with the two unknowns vm and f we can solve for vm, note that we put (-vm) for the approach. this leaves vm = v (f'1 - f'2)/(f'1+f'2) and with the numbers inserted we get vm = 104 m/s. but we must correct for altitude and get 104/sin(45) = 147m/s which equals 530 km/h.*